

Reviewer Pack — Sandpile Group Order Lower-Bounds Monotone KW Depth

Entry f75d64a69735 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-17 05:45:32 UTC

Sandpile Group Order Lower-Bounds Monotone KW Depth

Entry ID: f75d64a69735 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-17 05:45:32 UTC

1. Conjecture

Field A (mathematical branch): Abelian sandpile / chip-firing group theory (Dhar 1990, Bjorner-Lovasz-Shor 1991; the critical/sandpile group $K(G)$ = coker of the reduced graph Laplacian, $|K(G)|$ = number of spanning trees by Kirchhoff). This corner of algebraic combinatorics / statistical mechanics has essentially no presence in monotone circuit lower-bound literature. **Field B** (complexity object): Monotone Karchmer-Wigderson protocol depth $D_{KW}(f) = \log_2$ of minimum monotone formula leaf size for a monotone Boolean function f , accessed via the bipartite KW relation (minterm, maxterm) $\rightarrow$ intersection coordinate.

Statement:

For every nonconstant monotone Boolean $f: \{0,1\}^n \rightarrow \{0,1\}$ let $M(f)$ and $K(f)$ be its minterm and maxterm antichains (as subsets of $[n]$). Form the bipartite multigraph $G(f)$ on vertex set $M(f) \sqcup K(f)$ with an edge of multiplicity $|m \cap k|$ between each $m \in M(f)$ and $k \in K(f)$, add a ground vertex g joined by a single edge to every $k \in K(f)$, let $\tau(f)$ = number of spanning trees of $(G(f) \cup \{g\})$ computed via the matrix-tree theorem on the reduced Laplacian, and put $\sigma(f) = \log_2(1 + \tau(f))$. Then $\sigma(f) \leq D_{KW}(f) \cdot (|M(f)| + |K(f)| + 1) \cdot \log_2(n+2)$, with the constant 1 in front being the conjectured tight prefactor. A single monotone f exhibiting $\sigma(f)$ strictly above this bound (using any verified upper bound on $D_{KW}(f)$ from an explicit protocol) falsifies the conjecture.

Rationale (proposer's reasoning):

The sandpile group of the KW bipartite intersection multigraph counts independent 'cycles of certificates' that any KW protocol must resolve; each protocol step (a coordinate query at an internal node) can collapse at most $O((|M| + |K|) \log n)$ such independent cycles by a Cheeger-style averaging on the Laplacian, suggesting depth lower-bounds via $\log|K(G)|$. Because $\tau(f)$ is determined by the BOTH-sided minterm/maxterm relation rather than a single-number truth-table statistic, the invariant is relational and exponential-time in the truth table, so it sidesteps the Razborov-Rudich natural-proofs barrier and the ring-only algebrization trap.

Taxonomy category: KARCHMER_WIGDERSON (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): cd4a73bca73bd449

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across all 30 random seeds ($n \in \{4..8\}$, 2..6 minterms) plus the 5 fixed benchmarks (THR_2^5, THR_3^6, MAJ_5, MAJ_7, PM_{3,3}), the inequality $\sigma(f) \leq D_{KW}(f) \cdot (|M(f)| + |K(f)| + 1) \cdot \log_2(n+2)$ must hold on every instance with zero violations.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.92	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.78	SAFE	SAFE
ALGEBRIZATION	SAFE	0.83	SAFE	SAFE
KARP_LIPTON	SAFE	0.92	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Karchmer Wigderson monotone formula depth spanning trees Laplacian - sandpile group chip-firing Boolean circuit complexity lower bound - matrix-tree theorem minterm maxterm bipartite monotone communication complexity

Top relevant hits considered: - [http://arxiv.org/abs/2107.05128v2] Karchmer-Wigderson Games for Hazard-free Computation - [http://arxiv.org/abs/1012.2478v2] Packing spanning trees and the k -tree protocol - [http://arxiv.org/abs/0904.3087v4] Distributed Maintenance of Anytime Available Spanning Trees in Dynamic Networks - [http://arxiv.org/abs/1606.05050v1] Proof Complexity Lower Bounds from Algebraic Circuit Complexity - [http://arxiv.org/abs/1904.05483v2] Parallels Between Phase Transitions and Circuit Complexity? - [http://arxiv.org/abs/2504.19966v3] Quantum circuit lower bounds in the magic hierarchy - [http://arxiv.org/abs/1102.2932v2] Applications of Monotone Rank to Complexity Theory - [http://arxiv.org/abs/2007.09501v3] A family of matrix-tree multijections

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.1s

5.1 Generated Python source

```
import sys
import random
import math
import itertools
from fractions import Fraction

def generate_random_monotone_function(n, num_minterms, seed):
    random.seed(seed)
    minterms = []
    for _ in range(num_minterms):
        m = tuple(random.randint(0, 1) for _ in range(n))
        minterms.append(m)
```

```

M = set(minterms)
K = set()
for m in M:
    for k in itertools.product([0, 1], repeat=n):
        if all(m[i] <= k[i] for i in range(n)):
            K.add(k)
return M, K

def build_bipartite_graph(M, K):
    n = len(next(iter(M))) if M else 0
    ground_vertex = tuple([2] * n)
    vertices = list(M) + list(K) + [ground_vertex]
    edges = {}
    for m in M:
        for k in K:
            if all(m[i] <= k[i] for i in range(n)):
                edges[(m, k)] = edges.get((m, k), 0) + 1
            edges[(k, ground_vertex)] = edges.get((k, ground_vertex), 0) + 1
    return vertices, edges

def matrix_mult(A, B):
    result = [[0 for _ in range(len(B[0]))] for _ in range(len(A))]
    for i in range(len(A)):
        for j in range(len(B[0])):
            for k in range(len(B)):
                result[i][j] += A[i][k] * B[k][j]
    return result

def matrix_sub(A, B):
    return [[A[i][j] - B[i][j] for j in range(len(A[0]))] for i in range(len(A))]

def matrix_transpose(A):
    return [[A[j][i] for j in range(len(A))] for i in range(len(A[0]))]

def matrix_minor(A, i, j):
    return [row[:j] + row[j+1:] for row in (A[:i] + A[i+1:])]

def matrix_determinant(A):
    n = len(A)
    if n == 1:
        return A[0][0]
    if n == 2:
        return A[0][0] * A[1][1] - A[0][1] * A[1][0]
    det = 0
    for j in range(n):
        det += ((-1) ** j) * A[0][j] * matrix_determinant(matrix_minor(A, 0, j))
    return det

def compute_spanning_trees(vertices, edges):
    n = len(vertices)
    if n <= 1:
        return 1
    laplacian = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            if i == j:

```

```

        laplacian[i][j] = sum(edges.get((vertices[i], vertices[k]), 0) for k in range(n) if k != i)
    else:
        laplacian[i][j] = -edges.get((vertices[i], vertices[j]), 0)
    reduced_laplacian = [row[1:] for row in laplacian[1:]]
    det = matrix_determinant(reduced_laplacian)
    return det

def compute_sigma(M, K):
    vertices, edges = build_bipartite_graph(M, K)
    tau = compute_spanning_trees(vertices, edges)
    sigma = math.log2(1 + tau)
    return sigma

def compute_dkw(M, K):
    n = len(next(iter(M))) if M else 0
    if not M or not K:
        return 0
    min_depth = float('inf')
    for m in M:
        for k in K:
            if all(m[i] <= k[i] for i in range(n)):
                depth = sum(1 for i in range(n) if m[i] != k[i])
                if depth < min_depth:
                    min_depth = depth
    return min_depth

def run_trial(seed):
    random.seed(seed)
    n = random.choice([4, 5, 6, 7, 8])
    num_minterms = random.randint(2, 6)
    M, K = generate_random_monotone_function(n, num_minterms, seed)
    sigma = compute_sigma(M, K)
    dkw = compute_dkw(M, K)
    bound = dkw * (len(M) + len(K) + 1) * math.log2(n + 2)
    conjecture_holds = sigma <= bound
    counterexample = "" if conjecture_holds else f"sigma={sigma} > bound={bound}"
    return {
        "metric_name": "sigma",
        "metric_value": sigma,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

def main():
    seeds = [int(arg) for arg in sys.argv[1:]] if len(sys.argv) > 1 else [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97]
    results = []
    for seed in seeds:
        result = run_trial(seed)
        result["seed"] = seed
        print(f"TRIAL: {result}")
        results.append(result)
    metric_values = [r["metric_value"] for r in results]
    mean = sum(metric_values) / len(metric_values)
    std = math.sqrt(sum((x - mean) ** 2 for x in metric_values) / len(metric_values))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

```

```

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean} std={std} support_fraction={support_fraction}")
else:
    failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
    counterexample = next(r["counterexample"] for r in results if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample={counterexample} first_failing_seed={failing_seed}")

if __name__ == "__main__":
    main()

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out after 240s (returncode 124) without producing a RESULT line, so the pre-registered support condition cannot be evaluated and the critic challenge stands. | next: Optimize the test harness (e.g., cache D_KW upper bounds, reduce seed count or cap n at 6) so all 35 instances complete within the time budget.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	168513
2	preregistration	claude_max	opus	0	0	5362
3	novelty	claude_max	opus	0	0	2859
4	novelty	claude_max	opus	0	0	8555
5	test_gen	mistral	codestral-latest	0	0	312515
6	test_gen	mistral	codestral-latest	0	0	323290
7	test_gen	mistral	codestral-latest	0	0	310850
8	test_gen	mistral	codestral-latest	0	0	308729
9	judge	claude_max	opus	0	0	4574

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 1445246 ms total latency. Provider mix: {'claude_max': 5, 'mistral': 4}

(full prompt+response transcripts available in [research/audit/f75d64a69735.jsonl](#))

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/f75d64a69735.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/f75d64a69735.tar.gz` (if generated)